qsBye

折腾笔记[18]-使用rust读取rosbag数据

摘要

使用rust读取rosbag数据(ROS1格式).

关键词

rust;ros;rosbag;

关键信息

项目地址
[https://github.com/ByeIO/bye.rosette.rs/blob/rust/crates/rosette_core/examples/rosbag1_read_fast_livo2.rs]

```
[workspace]
version = "0.0.1"
authors = ["qsbye <2557877116@qq.com"]
members = [ "crates/rosette_cli","crates/rosette_core", "crates/rosette_derive", "crates/rose
resolver = "3"

[workspace.package]
name = "bye_rosette_rs"
version = "0.0.1"
authors = ["qsbye <2557877116@qq.com"]
repository = "https://github.com/ByeIO/bye.rosette.rs"
edition = "2024"
license = "Apache-2.0"

[workspace.dependencies]
# 错误处理
anyhow = "1.0.97"
# 时间格式化
chrono = "0.4.40"
# Model Context Protocol协议定义
rust-mcp-schema = { version = "0.2.2", path = "./static/rust-mcp-schema" }
# Model Context Protocol开发工具
rust-mcp-sdk = { version = "0.1.2", path = "./static/rust-mcp-sdk/crates/rust-mcp-sdk" }
# 多线程框架
tokio = { version = "1.44.1", features = ["full"] }
# ros2接口(针对humble)
ros2-interfaces-humble = { version = "0.0.1", path = "./static/ros2-interfaces-humble" }
# 日志后端
log = { version = "0.4.27", features = ["std"] }
# ros2数据包bag处理
rosbag2-rs = { version = "0.2.1", path = "./static/rosbag2-rs" }
# ros1数据包bag处理
rosbag = { version = "0.6.3", path = "./static/rosbag-rs" }
# 临时文件
tempfile = { version = "3.19.1" }
# ros2消息收发
ros2-client = { version = "0.8.1", features = ["pre-iron-gid"] }
# DDS分布式通信
rustdds = { version = "0.11.4", path = "./static/RustDDS" }
# MQTT协议栈(纯Rust)
rumqttc = { version = "0.24.0", path = "./static/rumqttc" }
rumqttd = { version = "0.19.0", path = "./static/rumqttd" }
# 静态变量
static_cell = "2.1.0"
# 全局变量
once_cell = "1.21.3"
# 通用数据表达序列化/反序列化
cdr = { version = "0.2.4", path = "./static/cdr-rs" }
# 大小端字节序
byteorder = "1.5.0"
```

原理简介

rosbag1格式对比rosbag2格式

[https://blog.csdn.net/lu_embedded/article/details/132451852]
[https://zhuanlan.zhihu.com/p/494474804]
以下是 ROS1 和 ROS2 中 rosbag 格式的对比：

- 文件结构
  ROS1 rosbag：以单个 .bag 文件存储所有数据，文件中包含消息的二进制数据、时间戳、主题信息等。
  ROS2 rosbag：采用目录结构，包含多个文件，如 .db3 数据库文件和 .yaml 配置文件。.db3 文件基于 SQLite，用于存储消息数据，而 .yaml 文件则存储元数据。
- 存储格式
  ROS1 rosbag：基于二进制格式存储数据，消息以二进制形式写入文件，每个消息包含时间戳和消息体。这种格式存储效率高，但在跨平台兼容性和扩展性方面存在限制。
  ROS2 rosbag：基于 SQLite 数据库格式，将消息存储为表格形式，每个表格包含时间戳、消息类型和数据。这种格式支持更灵活的查询和过滤操作。
- 兼容性
  ROS1 rosbag：由于其二进制格式的固定性，不支持跨平台的消息兼容性。
  ROS2 rosbag：具有更好的跨平台兼容性，支持多种操作系统和架构。
- 扩展性
  ROS1 rosbag：格式相对固定，不支持动态扩展。
  ROS2 rosbag：格式灵活，支持动态扩展，能够适应不同类型和结构的消息。
- 查询与过滤
  ROS1 rosbag：查询和过滤功能相对简单，主要通过主题名称、时间范围等进行过滤。
  ROS2 rosbag：由于基于数据库，支持更复杂的查询和过滤操作，用户可以更灵活地检索特定消息。
- 转换工具
  ROS1 rosbag：需要借助工具如 rosbag_migration_tool 或 rosbags-convert 将其转换为 ROS2 格式。
  ROS2 rosbag：同样可以使用上述工具将其转换为 ROS1 格式。

cdr序列化/反序列化简介

[https://linuxcpp.0voice.com/?id=232477]
CDR（Common Data Representation）是一种用于序列化和反序列化数据的格式，主要在分布式系统中应用，以便于不同平台和编程语言之间的数据交换。在RTPS（Real-Time Publish-Subscribe Protocol）以及DDS（Data Distribution Service）等协议中，CDR 被广泛使用来处理消息的传输。

CDR（Common Data Representation）序列化和反序列化是一种用于数据交换的技术，广泛应用于分布式系统和通信协议中，特别是在DDS（Data Distribution Service）和RTPS（Real-Time Publish-Subscribe Protocol）等场景。

CDR序列化

CDR序列化是将数据结构（如结构体、类等）转换为字节流的过程，以便通过网络传输或存储到文件中。CDR定义了一种平台无关的二进制格式，支持多种数据类型（如整数、浮点数、字符串、结构体等），并提供相应的编码规则。在序列化过程中，数据按照预定格式写入字节流，同时需要考虑字节对齐和大小端转换等问题。

CDR反序列化

CDR反序列化是序列化的逆过程，即将字节流还原为原始数据结构。接收方根据CDR规范解析字节流，按照数据类型和格式逐步读取字段，并重新构造出原始数据。

CDR的特点

- **跨平台和跨语言**：CDR支持多种编程语言（如C++、Java、Python等）之间的数据交换。
- **高效性**：CDR提供高效的序列化和反序列化方法，适合高性能通信场景。
- **灵活性**：支持复杂数据类型，包括嵌套结构和变长数据类型。
- **兼容性**：CDR支持版本管理，允许在不破坏现有二进制兼容性的情况下更改数据格式。

应用场景

CDR广泛应用于以下领域：

- **实时通信系统**：如自动驾驶、实时监控等，需要高效数据传输。
- **分布式系统**：用于不同组件之间的数据交换。
- **物联网**：在嵌入式设备之间传输数据。

通过CDR序列化和反序列化，可以实现跨平台、跨语言的数据交换，满足分布式系统和实时通信的需求。

实现

```
#![allow(unused)]

//! 读取fast_livo2示例rosbag

// 错误处理
use anyhow::{Result, Ok};

// rosbag1处理
use rosbag::{ChunkRecord, MessageRecord, RosBag, record_types::{Chunk, IndexData}};

// 字节处理
use byteorder::{LittleEndian, ReadBytesExt};

// 标准库
use std::path::Path;
use std::env;
use std::collections::HashMap;
use std::io::Cursor;
use std::io::prelude::*;

#[derive(Debug)]
struct Stamp {
    secs: i32,
    nsecs: i32,
}

#[derive(Debug)]
struct Header {
    seq: u32,
    stamp: Stamp,
    frame_id: String,
}

#[derive(Debug)]
struct Point {
    offset_time: u32,
    x: f32,
    y: f32,
    z: f32,
    reflectivity: u8,
    tag: u8,
    line: u8,
}

#[derive(Debug)]
struct CustomMsg {
    header: Header,
    timebase: u64,
    point_num: u32,
    lidar_id: u8,
    rsvd: [u8; 3],
    points: Vec<Point>,
}

fn parse_custom_msg(data: &[u8]) -> Result<CustomMsg> {
    let mut cursor = Cursor::new(data);
    let seq = cursor.read_u32::<LittleEndian>()?;
    let secs = cursor.read_i32::<LittleEndian>()?;
    let nsecs = cursor.read_i32::<LittleEndian>()?;
    let frame_id_len = cursor.read_u32::<LittleEndian>()? as usize;
    let mut frame_id_bytes = vec![0u8; frame_id_len];
    cursor.read_exact(&mut frame_id_bytes)?;
    let frame_id = String::from_utf8(frame_id_bytes)?;
    let timebase = cursor.read_u64::<LittleEndian>()?;
    let point_num = cursor.read_u32::<LittleEndian>()?;
    let lidar_id = cursor.read_u8()?;
    let mut rsvd = [0u8; 3];
    cursor.read_exact(&mut rsvd)?;
    let mut points = Vec::with_capacity(point_num as usize);
    for _ in 0..point_num {
        points.push(Point {
            offset_time: cursor.read_u32::<LittleEndian>()?,
            x: cursor.read_f32::<LittleEndian>()?,
            y: cursor.read_f32::<LittleEndian>()?,
            z: cursor.read_f32::<LittleEndian>()?,
            reflectivity: cursor.read_u8()?,
            tag: cursor.read_u8()?,
            line: cursor.read_u8()?,
        });
    }
    Ok(CustomMsg {
        header: Header { seq, stamp: Stamp { secs, nsecs }, frame_id },
        timebase,
        point_num,
        lidar_id,
        rsvd,
        points,
    })
}

/// 处理单个数据块中的消息记录
fn process_chunk(
    chunk: Chunk,
    conn_id_to_topic: &mut HashMap<u32, String>,
    message_count: &mut usize,
) -> Result<()> {
    for msg_record in chunk.messages() {
        match msg_record? {
            MessageRecord::Connection(conn) => {
                conn_id_to_topic.insert(conn.id, conn.topic.to_ascii_lowercase());
            }
            MessageRecord::MessageData(msg_data) => {
                if let Some(topic) = conn_id_to_topic.get(&msg_data.conn_id) {
                    if topic == "/livox/lidar" {
                        let custom_msg = parse_custom_msg(&msg_data.data)?;

                        // 结构化输出
                        println!("header: ");
                        println!("  seq: {}", custom_msg.header.seq);
                        println!("  stamp: ");
                        println!("    secs: {}", custom_msg.header.stamp.secs);
                        println!("    nsecs: {}", custom_msg.header.stamp.nsecs);
                        println!("  frame_id: \"{}\"", custom_msg.header.frame_id);
                        println!("timebase: {}", custom_msg.timebase);
                        println!("point_num: {}", custom_msg.point_num);
                        println!("lidar_id: {}", custom_msg.lidar_id);
                        println!("rsvd: [{}, {}, {}]", custom_msg.rsvd[0], custom_msg.rsvd[1],
                        println!("points: ");
                        for point in &custom_msg.points {
                            println!("  - ");
                            println!("    offset_time: {}", point.offset_time);
                            println!("    x: {:.15}", point.x);
                            println!("    y: {:.15}", point.y);
                            println!("    z: {:.15}", point.z);
                            println!("    reflectivity: {}", point.reflectivity);
                            println!("    tag: {}", point.tag);
                            println!("    line: {}", point.line);
                        }
                        println!("\n-----------------------------");

                        *message_count += 1;
                        if *message_count >= 5 {
                            return Ok(());
                        }
                    }
                }
            }
        }
    }
    Ok(())
}

fn main() -> Result<()> {
    let bag_relative_path = "../../assets/Retail_Street.bag";
    let current_dir = env::current_dir().expect("获取当前目录失败");
    let bag_abs_path = current_dir.join(bag_relative_path);
    println!("正在打开ROS bag文件: {}", bag_abs_path.display());

    let bag = RosBag::new(bag_abs_path)?;
    let mut conn_id_to_topic = HashMap::new();
    let mut message_count = 0;

    for chunk_record in bag.chunk_records() {
        match chunk_record? {
            ChunkRecord::Chunk(chunk) => {
                // 处理当前数据块
                process_chunk(chunk, &mut conn_id_to_topic, &mut message_count)?;
                if message_count >= 5 {
                    break;
                }
            }
            ChunkRecord::IndexData(_) => {}
        }
    }

    println!("成功读取前5条消息");
    Ok(())
}
```

运行:

```
cargo run --example rosbag1_read_fast_livo2
```

效果

原始rostopic输出:

```
header:
  seq: 4068
  stamp:
    secs: 946685438
    nsecs: 699309945
  frame_id: "livox_frame"
timebase: 946685278699310000
point_num: 24000
lidar_id: 0
rsvd: [0, 0, 0]
points:
  -
    offset_time: 0
    x: 15.404000282287598
    y: 5.798999786376953
    z: 5.0370001792907715
    reflectivity: 15
    tag: 16
    line: 0
  -
    offset_time: 4167
    x: 15.5
    y: 5.89300012588501
    z: 5.381999969482422
    reflectivity: 17
    tag: 16
    line: 1
  -
    offset_time: 8334
    x: 15.468000411987305
    y: 5.941999912261963
    z: 5.690999984741211
    reflectivity: 19
    tag: 16
    line: 2
  -
    offset_time: 12501
    x: 15.496999740600586
    y: 6.019000053405762
    z: 6.0320000648498535
    reflectivity: 19
    tag: 16
    line: 3
  -
    offset_time: 16668
    x: 15.512999534606934
    y: 6.0980000495910645
    z: 6.377999782562256
    reflectivity: 18
    tag: 16
    line: 4
  -
```

代码输出:

```
header:
  seq: 4056
  stamp:
    secs: 946685437
    nsecs: 499511003
  frame_id: "livox_frame"
timebase: 946685277499511000
point_num: 24000
lidar_id: 0
rsvd: [0, 0, 0]
points:
    -
    offset_time: 83886080
    x: 0.000000000000000
    y: 12.005999565124512
    z: 2.029999971389771
    reflectivity: 25
    tag: 4
    line: 210
    -
    offset_time: 1056064
    x: 0.000000000000000
    y: 12.024000167846680
    z: 2.036999940872192
    reflectivity: 225
    tag: 122
    line: 220
    -
    offset_time: 17834048
    x: 0.000000000000000
    y: 11.967000007629395
    z: 2.033999919891357
    reflectivity: 25
    tag: 4
    line: 230
    -
```

分类: 机器人 , 折腾笔记
标签: Rust

好文要顶 关注我 收藏该文 微信分享

qsBye
粉丝 - 27 关注 - 0
会员号：107

0 推荐 0 反对

« 上一篇： 折腾笔记[17]-使用rust创建linux系统服务

posted @ 2025-04-03 08:59 qsBye 阅读(0) 评论(0) MD 编辑 收藏 举报

刷新评论 刷新页面 返回顶部

发表评论

编辑 预览

支持 Markdown

自动补全

提交评论 退出

[Ctrl+Enter快捷键提交]

【推荐】100%开源！大型工业跨平台软件C++源码提供，建模，组态！
【推荐】还在用 ECharts 开发大屏？试试这款永久免费的开源 BI 工具！
【推荐】国内首个AI IDE，深度理解中文开发场景，立即下载体验Trae
【推荐】编程新体验，更懂你的AI，立即体验豆包MarsCode编程助手
【推荐】轻量又高性能的 SSH 工具 IShell：AI 加持，快人一步

编辑推荐：
· 继承的思维：从思维模式到架构设计的深度解析
· 如何在 .NET 中 使用 ANTLR4
· 后端思维之高并发处理方案
· 理解Rust引用及其生命周期标识（下）
· 从二进制到误差：逐行拆解C语言浮点运算中的4008175468544之谜

阅读排行：
· Cursor预测程序员行业倒计时：CTO应做好50%裁员计划
· 大模型 Token 究竟是啥：图解大模型Token
· 用99元买的服务器搭一套CI/CD系统
· 当职场成战场：降职、阴谋与一场硬碰硬的抗争
· HttpClient使用方法总结及工具类封装

公告

昵称： qsBye
园龄： 8年5个月
粉丝： 27
关注： 0

2025年4月

日	一	二	三	四	五	六
30	31	1	2	3	4	5
6	7	8	9	10	11	12
13	14	15	16	17	18	19
20	21	22	23	24	25	26
27	28	29	30	1	2	3
4	5	6	7	8	9	10

搜索

常用链接

我的随笔
我的评论
我的参与
最新评论
我的标签

我的标签

Rust(14)
SLAM(5)
线性代数(2)
LLM(1)

随笔分类

51单片机(10)
Android开发(18)
bouffalo单片机(1)
esp32单片机(24)
GNU Radio(4)
LLM精彩问答(7)
MATLAB(6)
RaspberryPi单片机(6)
SCILAB(3)
stm32单片机(16)
TI单片机(3)
WebAssembly(3)
翻译(5)
高云FPGA(3)
公众号(2)
鸿蒙开发(3)
机器人(12)
前端开发(12)
嵌入式Linux(3)
人工智能(4)
运维相关(16)
折腾笔记(20)

随笔档案

2025年4月(1)
2025年3月(4)
2025年2月(6)
2025年1月(11)
2024年12月(1)
2024年10月(1)
2024年9月(4)
2024年8月(2)
2024年6月(3)
2024年5月(9)
2024年4月(3)
2024年3月(10)
2024年2月(8)
2024年1月(9)
2023年12月(4)
2023年11月(6)
2023年10月(4)
2023年9月(4)
2023年8月(10)
2023年7月(16)
2023年6月(2)
2023年5月(2)
2023年4月(3)
2023年3月(2)
2023年2月(7)
2023年1月(17)
2022年12月(11)
2022年11月(14)
2022年10月(8)
2022年9月(17)
更多

阅读排行榜

1. OpenWrt安装Filebrowser网页文件管理器(11617)
2. esp32笔记[5]-基于I2S协议实现音频播放(9529)
3. 谈谈NAS的硬盘SATA背板(6717)
4. 三星平板的笔记模板(比例:16:10)(4323)
5. 电犀牛R66s软路由体验(4274)

评论排行榜

1. 51单片机笔记[1]-基础实验(1)
2. zemax光学设计笔记[5]-ZPL或ZOS-API的简单使用(1)
3. 小米6安装Ubuntu Touch(1)

推荐排行榜

1. 折腾笔记[15]-配置Zed编辑器(1)
2. esp32笔记[5]-基于I2S协议实现音频播放(1)
3. esp32笔记[1]-blink闪灯(1)
4. stm32笔记[2]-HAL库驱动2.4寸屏幕(1)
5. 基于esp32的MQTT开关门检测模块(1)

最新评论

1. Re:zemax光学设计笔记[5]-ZPL或ZOS-API的简单使用
请问博主用vsstudio运行的时候无法读取类型库文件(ZOSAPI.tlb)如何解决啊，ZOSAPII.tlb能用import读取，ZOSAPI.tlb读取却报错，研究了一天了。这两个文件在zosa...
--赤Xin
2. Re:小米6安装Ubuntu Touch
您好！请问应该如何扩容docker呢？
我按照本文给出的方案安装之后docker容量只有4G
--VantHoff
3. Re:51单片机笔记[1]-基础实验
大学买的单片机就没打开过.
--mossxzzom

Copyright © 2025 qsBye
Powered by .NET 9.0 on Kubernetes